

Esquemas Iterativos y Recursivos con Secuencias y Acumuladores

**Análisis y Diseño de Datos y Algoritmos
Estructuras de Datos y Algoritmos**

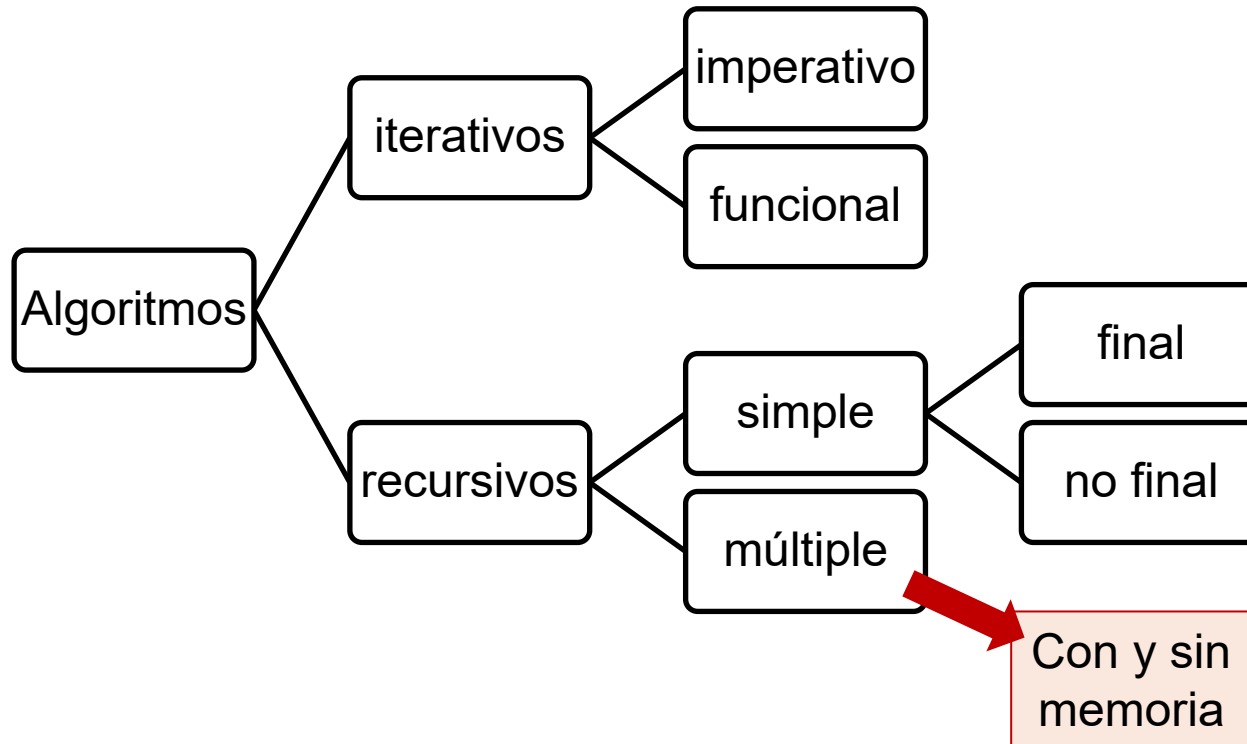
**ESCUELA TÉCNICA SUPERIOR DE
INGENIERÍA INFORMÁTICA
Departamento de Lenguajes y Sistemas Informáticos
Curso 2026-2027**

-
- ☐ Introducción
 - ☐ Imperativo, funcional y recursivo
 - ☐ Secuencias
 - Factorías de secuencias
 - ☐ Acumuladores
 - Factorías de acumuladores
 - ☐ Transformación de algoritmos iterativos y recursivos
 - ☐ Diseño de algoritmos

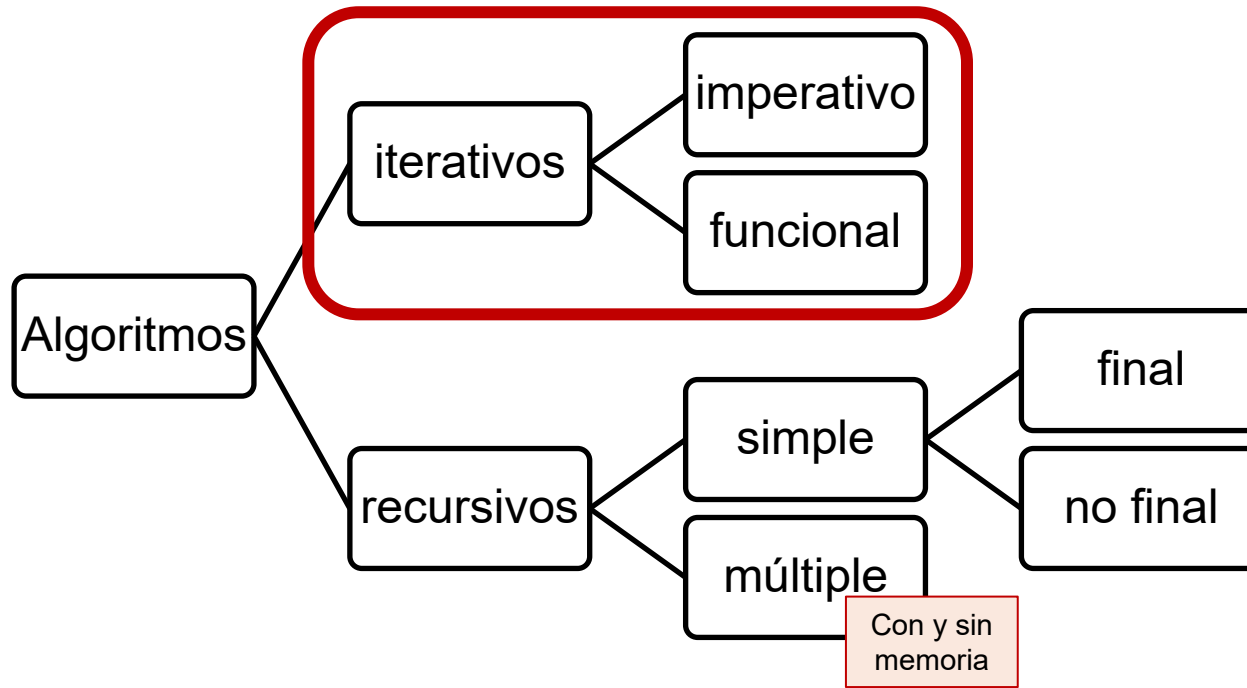
Introducción

- ❑ Rangos – $[a, b)$, $[a, b]$
- ❑ Tuplas – $t = (t_0, t_1, \dots, t_{n-1})$
- ❑ Listas – $ls = [e_0, e_1, \dots, e_{n-1}]$, $|ls|$, $ls[a,b]$, $ls[i]$, $[]$
- ❑ Conjuntos – $ss = \{e_0, e_1, \dots, e_{n-1}\}$, $|ss|$, $\{\}$
- ❑ Multiconjuntos – $ms = \{e_0:m_0, e_1:m_1, \dots, e_{n-1}:m_{n-1}\}$
- ❑ Diccionarios – $m = \{k_0:v_0, k_1:v_1, \dots, k_{n-1}:v_{n-1}\}$, $m[k_i]$
- ❑ Agregados de datos – d
- ❑ Secuencias – s

Clasificación de algoritmos

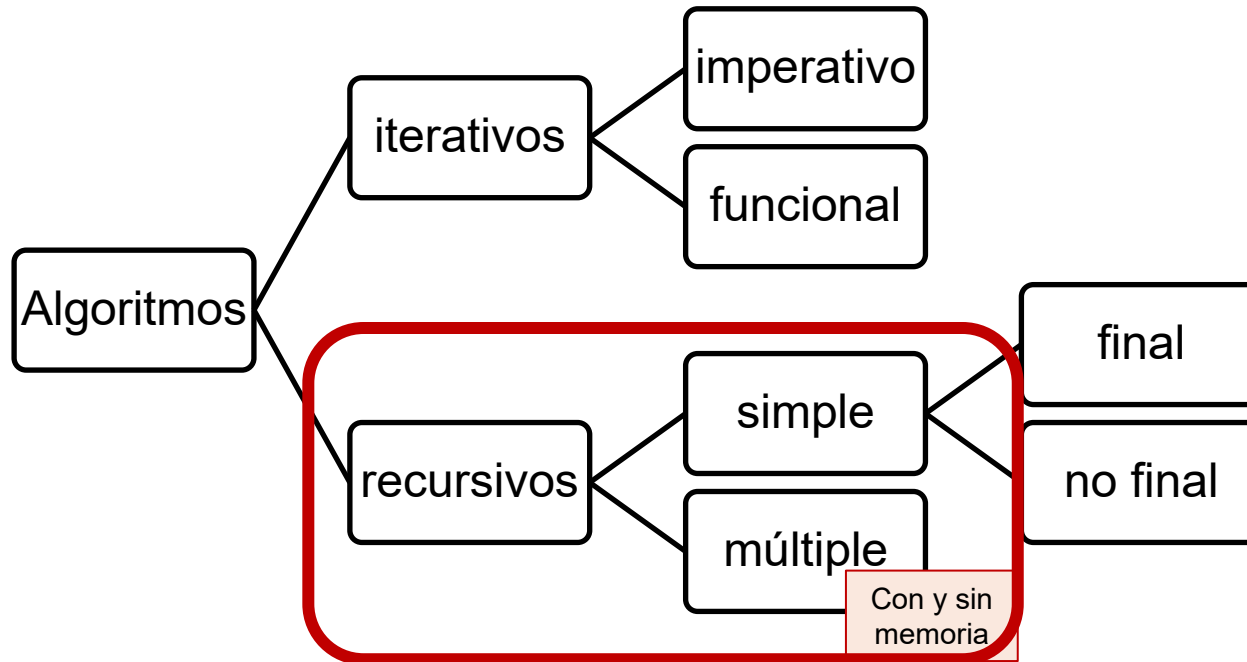


Clasificación de algoritmos



Instrucciones de manera repetida hasta que se cumple una condición de finalización

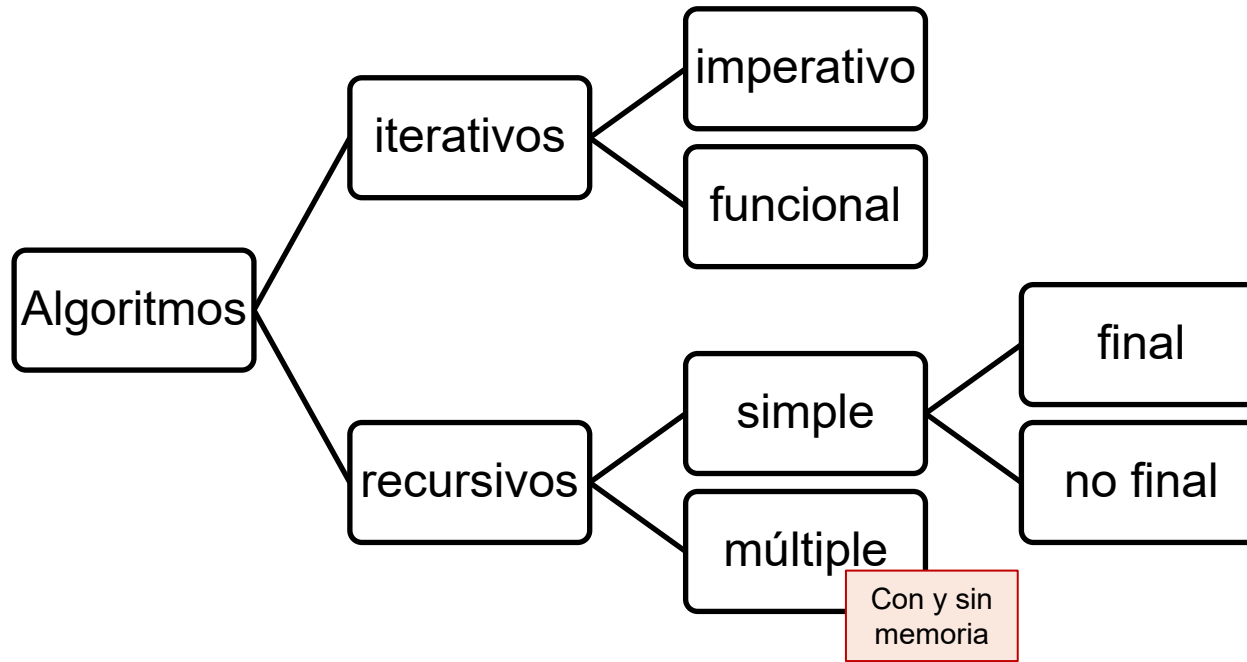
Clasificación de algoritmos



Dividir en problemas más pequeños → resuelve y combina.

Llamadas a sí mismos, cambiando parámetros, hasta llegar a caso base.

Clasificación de algoritmos



Haremos transformaciones entre ellos, cuando sea posible:

- imperativo $\leftrightarrow$ funcional
- iterativo $\leftrightarrow$ recursivo final
- no final $\rightarrow$ final (no siempre es posible)
- recursivo múltiple $\rightarrow$ iterativo

Algoritmos iterativos, ejemplo

Ejemplo imperativo

```
Double sumList(List<Double> list)) {  
    Integer i = 0;  
    Double b = 0.;  
    while (i < list.size()) {  
        b = b + list.get(i);  
        i = i + 1;  
    }  
    return b;  
}
```

Versión funcional

```
double sumListFun(List<Double> list) {  
    IntStream s = IntStream.range(0, list.size());  
    DoubleStream s2 = s.mapToDouble(i -> list.get(i));  
    return s2.sum();  
}
```

Algoritmos recursivos, ejemplo

```
Double sumListRec(List<Double> list) {
    return sumList(0, 0., list);
}

Double sumListRec(Integer i, Double b, List<Double> list) {
    if (i < list.size()) {
        Double e = list.get(i);
        i = i + 1;
        b = b + e;
        b = sumList(i, b, list);
    }
    return b;
}

Double r = sumListRec(List.of(3.,4.,11.,10.));
```

i	0	1	2	3	4	3	2	1.	0
b	0.	3.	7.	18.	28.	28.	28.	28.	28.
r									28.

Algoritmos recursivos

Escribimos los algoritmos recursivos como esquemas

$$n! = \begin{cases} 1, & n = 0 \\ n * (n - 1)!, & n > 0 \end{cases}$$

```
Integer f(Integer n) {  
    if (n==0) {  
        r = 1;  
    } else {  
        r = n * f(n-1);  
    }  
    return r;  
}
```

Algoritmos recursivos: tipos

Simple, múltiple. Final, no final

Algoritmo recursivo final:

$$\text{mcd}(a, b) = \begin{cases} a, & b = 0 \\ \text{mcd}(b, a \% b), & a, b > 0 \end{cases}$$

Algoritmo recursivo simple no final:

$$n! = \begin{cases} 1, & n = 0 \\ n * (n - 1)!, & n > 0 \end{cases}$$

Algoritmo recursivo múltiple:

$$\text{fib}(n) = \begin{cases} n, & 0 \leq n \leq 1 \\ \text{fib}(n - 1) + \text{fib}(n - 2), & n > 1 \end{cases}$$

Algoritmos iterativos

Algoritmos iterativos

Estructura clásica general

```
R f(x) {  
  E e = e0;  
  B b = b0;  
  while (g(e)) {  
    b = c(b, e);  
    e = nx(e);  
  }  
  return r(b);  
}
```

Esquema funcional (abstracto)

```
R f(x) {  
  S s = Stream.iterate(e0, e->g(e), e->nx(e));  
  Collector cl = (b0, (b,e)->c(b,e), b->r(b), ... ;  
  return s.collect(cl);  
}
```

Algoritmos iterativos

Estructura clásica general

```
R f(x) {  
  E e = e0;  
  B b = b0;  
  while (g(e)) {  
    b = c(b, e);  
    e = nx(e);  
  }  
  return r(b);  
}
```

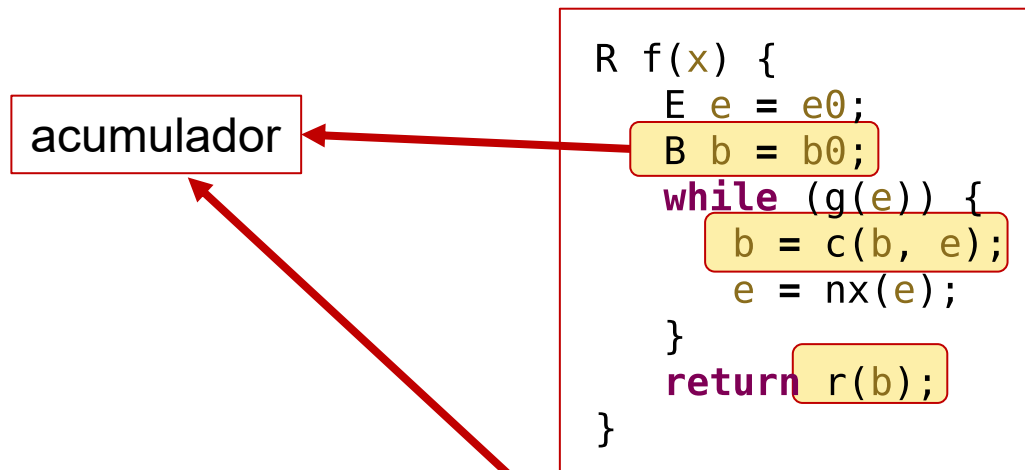
secuencia

Esquema funcional (abstracto)

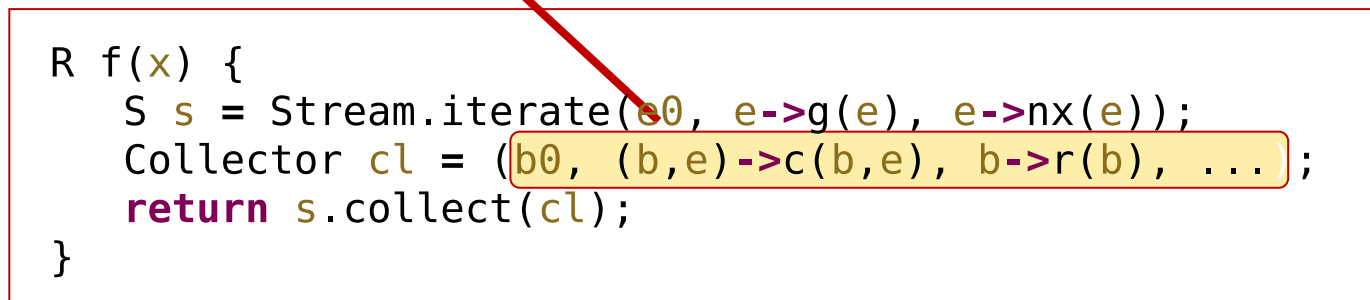
```
R f(x) {  
  S s = Stream.iterate(e0, e->g(e), e->nx(e));  
  Collector cl = (b0, (b,e)->c(b,e), b->r(b), ... ;  
  return s.collect(cl);  
}
```

Algoritmos iterativos

Estructura clásica general



Esquema funcional (abstracto)



Recursividad final y algoritmos iterativos

Recursividad final y algoritmos iterativos

Esquema iterativo

```
R f(x) {  
  E e = e0;  
  B b = b0;  
  while (g(e)) {  
    b = c(b, e);  
    e = nx(e);  
  }  
  return r(b);  
}
```

Esquema recursivo

```
R f(X x){  
  E e = e0;  
  B b = b0;  
  b = f(e, b, x);  
  return r(b);  
}  
  
B f(E e, B b, X x){  
  if(g(e)){  
    b = c(b, e);  
    e = nx(e);  
    b = f(e, b, x);  
  }  
  return b;  
}
```

Recursividad final y algoritmos iterativos

Esquema iterativo

```

R f(x) {
  E e = e0;
  B b = b0;
  while (g(e)) {
    b = c(b, e);
    e = nx(e);
  }
  return r(b);
}
    
```

Esquema recursivo

```

R f(X x){
  E e = e0;
  B b = b0;
  b = f(e, b, x);
  return r(b);
}

B f(E e, B b, X x){
  if(g(e)){
    b = c(b, e);
    e = nx(e);
    b = f(e, b, x);
  }
  return b;
}
    
```

secuencia

Recursividad final y algoritmos iterativos

Esquema iterativo

```

R f(x) {
  E e = e0;
  B b = b0;
  while (g(e)) {
    b = c(b, e);
    e = nx(e);
  }
  return r(b);
}
  
```

Esquema recursivo

```

R f(X x){
  E e = e0;
  B b = b0;
  b = f(e, b, x);
  return r(b);
}

B f(E e, B b, X x){
  if(g(e)){
    b = c(b, e);
    e = nx(e);
    b = f(e, b, x);
  }
  return b;
}
  
```

Acumulador

Secuencias

Implementación de secuencias

Son flujos secuenciales de elementos que se pueden recorrer uno detrás de otro

Algoritmo iterativo → tiene asociada una secuencia finita

Tipos más utilizados para su implementación:

- ❑ Stream<E> en Java
- ❑ Iterator<E> en Java

```
interface Iterator<E> {  
    boolean hasNext(); //Indica si hay más elementos  
    E next(); //Devuelve el siguiente y pone otro disponible  
    ...  
}
```

- ❑ Cada iterator se concreta en una clase que implementa Iterator<E>
- ❑ Podemos obtener un iterator de un stream con el método iterator()

Implementación de secuencias

```
public interface Iterable<E> { // Interfaz iterable
    public Iterator<E> iterator();
}

public class ArrayList implements List<E>, Iterable<E> {
    public Iterator<E> iterator() {
        return new ListIterator(this); // Clase interna
    }
}

List<E> ls = new ArrayList<>();
Iterator<E> it = ls.iterator();
while (it.hasNext()) { // Uso de un iterador
    E o = it.next();
    System.out.println(o);
}

for (E o: ls) { // for extendido
    System.out.println(o);
}
```

Implementación de secuencias

```
public class ListIterator<E> implements Iterator<E> {  
    private List<E> list;  
    private Integer i;  
    public ListIterator(List<E> list) {  
        this.list = list;  
        this.i = 0;  
    }  
  
    public boolean hasNext() {  
        return i < list.size();  
    }  
  
    public E next() {  
        E e = list.get(i);  
        i++;  
        return e;  
    }  
}
```

Factoría de secuencias

Rango: secuencia (enteros o reales) de a hasta $b \rightarrow [a, b)$

$$\text{range}(a, b) \equiv (a, e \rightarrow e < b, e \rightarrow e + 1)$$

Iterate: secuencia general, definida por valor inicial, predicado y operador unario.

$$\text{iterate}(e0, g, nx) \equiv (e0, e \rightarrow g(e), e \rightarrow nx(e))$$

Secuencias asociadas a agregados: el agregado exporta un iterador para poder recorrer los elementos que contiene.

En Java:

- `c.stream()` para colecciones
- `Files.lines(file)` para ficheros
- `text.chars()` para cadenas de caracteres
- `Arrays.stream(a)` para arrays
- ...

Factoría de secuencias

Ejemplos

```
Integer sumRangeFun(Integer a, Integer b) {  
    return IntStream.range(a,b).sum();  
}
```

```
Integer sumRangeWhile(Integer a, Integer b) {  
    Integer ac = 0;  
    Integer e = a;  
    while(e < b) {  
        ac = ac + e;  
        e = e + 1;  
    }  
    return ac;  
}
```

Factoría de secuencias

Ejemplos

```
Double sumGeometricRangeFun(Double a, Double b, Double c) {  
    Double e0 = a;  
    Predicate<Double> g = e -> e<b;  
    UnaryOperator<Double> nx = e -> e*c;  
    return Stream.iterate(e0, g, nx).mapToDouble(e->e).sum();  
}
```

```
Double sumGeometricRangeWhile(Double a, Double b, Double c) {  
    Double ac = 0;  
    Double e = a;  
    while (e<b) {  
        ac = ac + e;  
        e = e * c;  
    }  
    return ac;  
}
```

Operaciones sobre secuencias

- **Transformación:** $s2 = s1.map(t)$, siendo t una función, sustituye cada elemento de la secuencia por el valor obtenido al aplicar la función
- **Filtro:** $s2 = s1.filter(h)$, siendo h un predicado, elimina de la secuencia todos los elementos que no cumplan el predicado
- **Aplanamiento:** $s2 = s1.flatMap(seq)$, siendo seq una función que por cada elemento de la secuencia original obtiene una nueva secuencia, y sustituye los elementos de la secuencia por los valores de la nueva secuencia obtenidos al aplicar la función
- **Límite:** $s2 = s1.limit(n)$, obtiene otra secuencia con los primeros n elementos

Operaciones sobre secuencias

Ejemplos – Transformación

```
Double sumSquaresFun(Integer a, Integer b) {  
    return IntStream.range(a,b).mapToDouble(e->e*e).sum();  
}
```

```
Double sumSquaresWhile(Integer a, Integer b) {  
    Double ac = 0;  
    Integer e = a;  
    Integer t;  
    while (e<b) {  
        t = e * e;  
        ac = ac + t;  
        e = e + 1;  
    }  
    return ac;  
}
```

Operaciones sobre secuencias

Ejemplos – Filtro

```
Integer sumMultiplesFun(Integer a, Integer b, Integer m) {  
    return IntStream.range(a,b).filter(e->e%m==0).sum();  
}
```

```
Integer sumMultiplesWhile(Integer a, Integer b, Integer m) {  
    Integer ac = 0;  
    Integer e = a;  
    while (e<b) {  
        if (e%m==0) {  
            ac = ac + e;  
        }  
        e = e + 1;  
    }  
    return ac;  
}
```

Operaciones sobre secuencias

Ejemplos – Aplanamiento

```
Integer sumElementsFun(List<Set<Integer>> ls) {  
    return ls.stream()  
        .flatMap(e->e.stream())  
        .mapToInt(Integer::intValue)  
        .sum();  
}
```

```
Integer sumElementsWhile(List<Set<Integer>> ls) {  
    Integer ac = 0;  
    Integer i = 0;  
    while (i<ls.size()) {  
        Iterator<Integer> it = ls.get(i).iterator();  
        while (it.hasNext()) {  
            Integer e = it.next();  
            ac = ac + e;  
        }  
        i = i + 1;  
    }  
    return ac;  
}
```

Operaciones sobre secuencias

Ejemplos – Límite

```
Long sumFirstPrimesFun(Integer n) {  
    return Stream.iterate(2, e->true, e->Math2.siguientePrimo(e))  
        .limit(n)  
        .mapToLong(Long::longValue)  
        .sum();  
}
```

```
Long sumFirstPrimesWhile(Integer n) {  
    Long ac = 0L;  
    Integer i = 0;  
    Long e = 2L;  
    while (i < n) {  
        ac = ac + e;  
        e = Math2.siguientePrimo(e);  
        i = i + 1;  
    }  
    return ac;  
}
```

Operaciones sobre secuencias

- **Prefijo:** $s2 = s1. takeWhile(p)$, siendo p un predicado, prefijo más largo de elementos de $s1$ que cumplen p
- **Sufijo:** $s2 = s1. dropWhile(p)$, siendo p un predicado, sufijo de $s1$ que resulta cuando se elimina el prefijo anterior
- **Concatenar:** $concat(s1, s2)$
- **Enumerate:** $enumerate(s)$, secuencia formada por los pares de elementos contruidos por los elementos de s y su posición en la misma.
- **Pares consecutivos**
- **Producto cartesiano**
- **Zip**

Acumuladores

Acumuladores

Forma general de algoritmo iterativo y funcional

```
R f(x) {
  E e = e0;
  B b = b0;
  while (g(e) && !d(b)) {
    b = c(b, e);
    e = nx(e);
  }
  return r(b);
}
```

```
R f(x) {
  A a = (b0, (b, e) → c(b, e), b → r(b), b → d(b)) ;
  S s = (e0, e → g(e), e → nx(e));
  R r = s.collect(a);
  return r;
}
```

Acumulador secuencial

$a = (b_0, (b, e) \rightarrow c(b, e), b \rightarrow r(b), b \rightarrow d(b))$

Base del
acumulador
(valor inicial)

Función de
acumulación

Función de
retorno

Función de
cortocuito

Acumulador simple

$a = (b_0, (b, e) \rightarrow c(b, e))$

Factorías de acumuladores

❑ Método general para acumular: *collect*

- `s.allMatch(p)`
- `s.noneMatch(p)`
- `s.anyMatch(p)`
- `s.sum()`
- `s.count()`

All(p): $B = R = \text{Boolean}, a = (\text{true}, (b, e) \rightarrow p(e), b \rightarrow b, b \rightarrow !b)$

None(p): $B = R = \text{Boolean}, a = (\text{false}, (b, e) \rightarrow p(e), b \rightarrow !b, b \rightarrow b)$

Any(p): $B = R = \text{Boolean}, a = (\text{false}, (b, e) \rightarrow p(e), b \rightarrow b, b \rightarrow b)$

Sum(): $B = R = \text{Number}, a = (0, (b, e) \rightarrow b + e)$

Count(): $B = R = \text{Integer}, a = (0, (b, e) \rightarrow b + 1)$

Factorías de acumuladores

□ Método general para acumular: *collect*

- `s.collect(Collectors.joining(sp,pf,sf))`

Joining(sp,pf,sf):

$B = (String, Boolean), E = R = String$

$a = ((pf, true), ((b1, b2), e) \rightarrow c((b1, b2), e), (b1, b2) \rightarrow b1 + sf, b \rightarrow false)$

Con

$$c((b1, b2), e) = \begin{cases} (b1 + e, false), & b2 = true \\ (b1 + sp + e, false), & b2 = false \end{cases}$$

Factorías de acumuladores

- ❑ Método general para acumular: *collect*
 - `s.reduce(id,bo)`

Reduce(e0,bo): $B = E = R, a = (e0, (b, e) \rightarrow bo(b, e))$

Reduce(bo): $B = E, R = Optional<E>$

$a = ((?, true), ((b1, b2), e) \rightarrow c(b1, b2, e), (b1, b2) \rightarrow Opt.of(b1), b \rightarrow false)$

$$c(b1, b2, e) = \begin{cases} (e, false), & b2 \\ (bo(b1, e), false), & !b2 \end{cases}$$

Combinando secuencias y acumuladores

```
s <- (e0, e->g(e), e->nx(e))  
a <- (b0, (b,e)->c(b,e), b->r(b), b->d(b))
```

```
R seqCollectLeft(S s, A a) {  
  E e = s.e0;  
  B b = a.b0;  
  while (s.g(e) && !a.d(b)) {  
    b = a.c(b, e);  
    e = s.nx(e);  
  }  
  return a.r(b);  
}
```

Transformación de algoritmos iterativos y recursivos

Transformación de alg. iterativos

- Transformación de la notación funcional a la imperativa y viceversa
- Se trata de identificar la secuencia y el acumulador, y a partir de ahí escribir el algoritmo
- Ejemplo: algoritmo para decidir si un número es primo

```
boolean isPrimeFun(Long n) {  
    Long sqrt = (long) Math.sqrt((double)n);  
    return LongStream.rangeClosed(2, sqrt)  
        .noneMatch(x -> n % x == 0);  
}
```

Secuencia, acumulador
y predicado:

```
s = (2, e → e ≤ √n, e → e + 1)  
B = Boolean  
a = (false, (b, e) → p(e), b → !b, b → b)  
p(e) = Math2.esDivisible(n, e)
```

```
boolean isPrimeWhile(Long n) {  
    Long sqrt = (long) Math.sqrt((double)n);  
    Long e = 2L;  
    Boolean b = false;  
    while (e <= sqrt && !b) {  
        b = n % e == 0;  
        e = e + 1;  
    }  
    return !b;  
}
```

Recursividad final y algoritmos iterativos

Un algoritmo iterativo tiene un algoritmo recursivo final equivalente

```
R seqCollectLeftWhile(X x) {  
  E e = e0;  
  B b = b0;  
  while (g(e) && !d(b)) {  
    b = c(b, e);  
    e = nx(e);  
  }  
  return r(b);  
}
```

```
R seqCollectLeftRec(X x) {  
  B b = seqColletLeft(x, b0, e0);  
  return r(b);  
}  
  
R seqCollectLeftRec(X x, B b, E e) {  
  if (g(e) && !d(b)) {  
    b = c(b, e);  
    e = nx(e);  
    b = seqCollectLeft(x, b, e);  
  }  
  return b;  
}
```

Transformación de alg. iterativos

```
boolean isPrimeWhile(Long n) {  
    Long sqrt = (long)Math.sqrt((double)n);  
    Long e = 2L;  
    Boolean b = false;  
    while (e <= sqrt && !b) {  
        b = n % e == 0;  
        e = e + 1;  
    }  
    return !b;  
}
```

```
Boolean isPrimeRec(Long n) {  
    Long sqrt = (long)Math.sqrt((double)n);  
    Long e = 2L;  
    Boolean b = false;  
    b = isPrimeRec(e, b, sqrt, n);  
    return !b;  
}
```

```
Boolean isPrimeRec(Long e, Boolean b, Long sqrt, Long n) {  
    if (e <= sqrt && !b) {  
        b = n % e == 0;  
        e = e + 1;  
        b = isPrimeRec(e, b, sqrt, n);  
    }  
    return b;  
}
```

Transformación de alg. iterativos

Ejemplo, funcional a imperativo: Sumar los enteros contenidos en una lista

```
Integer sumListFun(List<Integer> ls) {  
    return ls.stream().mapToInt(x->x).sum();  
}
```

```
Integer sumListFor(List<Integer> ls) {  
    Integer b = 0;  
    for (Integer e: ls) {  
        b = b + e;  
    }  
    return b;  
}
```

```
Integer sumListWhile(List<Integer> ls) {  
    Integer b = 0;  
    Integer i = 0;  
    while (i < ls.size()) {  
        Integer e = ls.get(i);  
        b = b + e;  
        i = i + 1;  
    }  
    return b;  
}
```

Transformación de alg. iterativos

Ejemplo, funcional a imperativo: Sumar los enteros contenidos en una lista

```
Integer sumListWhile(List<Integer> ls) {  
    Integer b = 0;  
    Integer i = 0;  
    while (i < ls.size()) {  
        Integer e = ls.get(i);  
        b = b + e;  
        i = i + 1;  
    }  
    return b;  
}
```

```
Integer sumListRec(List<Integer> ls) {  
    Integer i = 0;  
    Integer b = 0;  
    b = sumListRec(i, b, ls);  
    return b;  
}
```

```
Integer sumListRec(Integer i, Integer b, List<Integer> ls) {  
    if (i < ls.size()) {  
        Integer e = ls.get(i);  
        b = b + e;  
        i = i + 1;  
        b = sumListRec(i, b, ls);  
    }  
    return b;  
}
```

Transformación de algoritmos

Ejemplo, funcional a imperativo: Dada una lista de enteros, comprobar si todos son impares

```
Boolean allOddFun(List<Integer> ls) {  
    return ls.stream().allMatch(x -> x%2 == 1);  
}
```

```
Boolean allOddFor(List<Integer> ls) {  
    Boolean b = true;  
    for (Integer e: ls) {  
        b = e%2 == 1;  
        if (!b) break;  
    }  
    return b;  
}
```

```
Boolean allOddWhile(List<Integer> ls) {  
    Boolean b = true;  
    Integer i = 0;  
    while (i < ls.size() && b) {  
        Integer e = ls.get(i);  
        b = e%2 == 1;  
        i = i + 1;  
    }  
    return b;  
}
```

Transformación de algoritmos

Ejemplo, funcional a imperativo: Dada una lista de enteros, comprobar si todos son impares

```
Boolean allOddWhile(List<Integer> ls) {  
    Boolean b = true;  
    Integer i = 0;  
    while (i < ls.size() && b) {  
        Integer e = ls.get(i);  
        b = e%2 == 1;  
        i = i + 1;  
    }  
    return b;  
}
```

```
Boolean allOddRec(List<Integer> ls) {  
    Integer i = 0;  
    Boolean b = true;  
    b = allOddRec(i, b, ls);  
    return b;  
}
```

```
Boolean allOddRec(Integer i, Boolean b, List<Integer> ls) {  
    if (i < ls.size() && b) {  
        Integer e = ls.get(i);  
        b = e%2 == 1;  
        i = i + 1;  
        b = allOddRec(i, b, ls);  
    }  
    return b;  
}
```

Diseño de algoritmos

Diseño de algoritmos iterativos

- **Diseñar un algoritmo iterativo** supone encontrar un algoritmo iterativo, imperativo o funcional, dada una restricción entrada-salida para el mismo
- Estrategia más sencilla:
 - Buscar un estado, una secuencia y el acumulador adecuado
 - Hay que demostrar que la secuencia es finita
 - El resultado debe ser la consecuencia de acumular la secuencia mediante el acumulador
 - Hay casos, los menos, dónde debemos escoger un estado y un invariante. Encontrar el estado se denomina generalizar el problema. A partir del estado y el invariante se deberá escoger la función siguiente para que se mantenga el invariante y la función de cota cumpla las propiedades exigidas.

Diseño de algoritmos iterativos

Ejemplo: Encontrar el siguiente primo de un número n

```
Long nextPrimeFun(Long n) {  
    Long e0 = n%2 == 0 ? n+1 : n+2;  
    return Stream.iterate(e0, e->e+2)  
        .filter(e -> Math2.esPrimo(e))  
        .findFirst()  
        .get();  
}
```

```
Long nextPrimeWhile(Long n) {  
    Long e = n%2 == 0 ? n+1 : n+2;  
    Long b = null;  
    while (b == null) {  
        if (Math2.esPrimo(e)) {  
            b = e;  
        }  
        e = e + 2;  
    }  
    return b;  
}
```

Diseño de algoritmos iterativos

```
Long nextPrimeWhile(Long n) {  
    Long e = n%2 == 0 ? n+1 : n+2;  
    Long b = null;  
    while (b == null) {  
        if (Math2.esPrimo(e)) {  
            b = e;  
        }  
        e = e + 2;  
    }  
    return b;  
}
```

Ejemplo: Encontrar el siguiente primo de un número n

```
Long nextPrimeRec(Long n) {  
    Long e = n%2 == 0 ? n+1 : n+2;  
    Long b = null;  
    b = nextPrimeRec(e, b);  
    return b;  
}
```

```
Long nextPrimeRec(Long e, Long b) {  
    if (b == null) {  
        if (Math2.esPrimo(e)) {  
            b = e;  
        }  
        e = e + 2;  
        b = nextPrimeRec(e, b);  
    }  
    return b;  
}
```

Diseño de algoritmos iterativos

Ejemplo: Obtener una lista con los primos menores que n

Ejemplo: Obtener la suma de los primos mayores o iguales que m y menores que n

Diseño de algoritmos iterativos

Ejemplo: Comprobar si una lista está ordenada

```
Boolean isListSortedFun(List<E> ls, Comparator<E> cmp) {  
    return IntStream.range(0, ls.size()-1)  
        .allMatch(i -> cmp.compare(ls.get(i), ls.get(i+1)) <= 0);  
}
```

```
Boolean isListSortedWhile(List<E> ls, Comparator<E> cmp) {  
    Integer i = 0;  
    Boolean b = true;  
    while(i < ls.size()-1 && b) {  
        b = cmp.compare(ls.get(i), ls.get(i+1)) <= 0;  
        i = i + 1;  
    }  
    return b;  
}
```

Diseño de algoritmos iterativos

```
Boolean isListSortedWhile(List<E> ls, Comparator<E> cmp) {  
    Integer i = 0;  
    Boolean b = true;  
    while(i < ls.size()-1 && b) {  
        b = cmp.compare(ls.get(i), ls.get(i+1)) <= 0;  
        i = i + 1;  
    }  
    return b;  
}
```

Ejemplo: Comprobar
si una lista está
ordenada

```
Boolean isListSortedRec(List<E> ls, Comparator<E> cmp) {  
    Integer i = 0;  
    Boolean b = true;  
    b = isListSortedRec(i, b, ls, cmp);  
    return b;  
}
```

```
Boolean isListSortedRec(Integer i, Boolean b, List<E> ls, Comparator<E> cmp) {  
    if (i < ls.size()-1 && b) {  
        b = cmp.compare(ls.get(i), ls.get(i+1)) <= 0;  
        i = i + 1;  
        b = isListSortedRec(i, b, ls, cmp);  
    }  
    return b;  
}
```

Diseño de algoritmos iterativos

Ejemplo: Obtener una lista de pares de un primo y su siguiente, tal que el primero sea mayor o igual que m y menor que n , y que la diferencia entre ellos sea una cantidad $k \geq 2$ dada

```
List<LongPair> primePairsFun(Long m, Long n, Integer k) {  
    Stream<Long> s = Stream.iterate(Math2.siguientePrimo(m-1),  
        x -> x<n, x -> Math2.siguientePrimo(x));  
    return Stream2.consecutivePairs(s)  
        .map(p -> LongPair.of(p))  
        .filter(t -> t.second() - t.first() == k)  
        .toList();  
}
```

Ejercicio propuesto: hacer la versión imperativa y recursiva

Diseño de algoritmos iterativos

Ejemplo: Obtener el número de veces que se repite cada divisor, mayor que 1, entre los divisores de los números que van de m a n

```
Stream<Long> divisors(Long n) {  
    return Stream.iterate(2L, x -> x <= (long)Math.sqrt(n), x -> x+1)  
        .filter(x -> n%x==0);  
}  
  
Multiset<Long> repetitionsDivisors(Long m, Long n) {  
    return Stream.iterate(m, x -> x <= n, x -> x+1)  
        .flatMap(x -> divisors(x))  
        .collect(Collectors2.toMultiset());  
}
```

Ejercicio propuesto: hacer la versión imperativa y recursiva

Diseño de algoritmos iterativos

Ejemplo: Búsqueda binaria: dada una lista ls , ordenada con respecto a un orden, encontrar, si existe, la posición de un elemento dado e o -1 si no lo encuentra

Generalizar problema

$$(i, j, k), i \in [0, n], j \in [0, n], k \in [0, n]$$

Invariante

$$e \in ls = e \in ls[i, j], i \leq j, k = \frac{i+j}{2}$$

Tamaño, estado inicial, guarda, siguiente, función de cota

$$s(i, j, k) = \begin{cases} (i, k, \frac{i+k}{2}), & e < ls[k] \\ (k+1, j, \frac{k+1+j}{2}), & e > ls[k] \end{cases}$$

Diseño de algoritmos iterativos

Ejemplo: Búsqueda binaria: dada una lista ls , ordenada con respecto a un orden, encontrar, si existe, la posición de un elemento dado e o -1 si no lo encuentra

```
Integer binarySearchWhile(List<E> ls, E e) {
    n = |ls|;
    (i, j, k) = (0, n, n/2);
    while (j-i > 0 && !(ls[k] == e)) {
        if (e < ls[k]) {
            (i, j, k) = (i, k, (i+k)/2);
        } else {
            (i, j, k) = (k+1, j, (k+1+j)/2);
        }
    }
    return j-i>0 ? k : -1;
}
```

$$E = (i, j, k), \quad sq = \left(\left(0, n, \frac{n}{2} \right), (i, j, k) \rightarrow j - i > 0, (i, j, k) \rightarrow s(i, j, k) \right)$$

$$B = E, \quad a = (?, ?, (i, j, k) \rightarrow j - i > 0 ? k : -1, (i, j, k) \rightarrow e = ls[k])$$

Diseño de algoritmos iterativos

```
Integer binarySearchWhile(List<E> ls, E e) {
    n = |ls|;
    (i, j, k) = (0, n, n/2);
    while (j-i > 0 && !(ls[k] == e)) {
        if (e < ls[k]) {
            (i, j, k) = (i, k, (i+k)/2);
        } else {
            (i, j, k) = (k+1, j, (k+1+j)/2);
        }
    }
    return j-i > 0 ? k : -1;
}
```

Ejemplo: Búsqueda binaria: dada una lista *ls*, ordenada con respecto a un orden, encontrar, si existe, la posición de un elemento dado *e* o -1 si no lo encuentra

```
Integer binarySearchRec(List<E> ls, E e) {
    n = |ls|;
    (i, j, k) = (0, n, n/2);
    (i, j, k) = binarySearchRec(i, j, k, ls, e);
    return j-i > 0 ? k : -1;
}
```

```
Tuple binarySearchRec(Integer i, Integer j, Integer k, List<E> ls, E e) {
    if (j-i > 0 && !(ls[k] == e)) {
        if (e < ls[k]) {
            (i, j, k) = (i, k, (i+k)/2);
        } else {
            (i, j, k) = (k+1, j, (k+1+j)/2);
        }
        (i, j, k) = binarySearchRec(i, j, k, ls, e);
    }
    return (i, j, k);
}
```

Diseño de algoritmos iterativos

Ejemplo: Bandera holandesa:

Dada una lista y un elemento del mismo tipo que las casillas, que llamaremos pivote, reordenarla, de menor a mayor, para que resulten tres bloques: los menores que el pivote, los iguales al pivote y los mayores que el pivote. El algoritmo debe devolver dos enteros que son las posiciones de las casillas que separan los tres bloques formados.

0	a	b	c	n
< p	= p	?	> p	

Diseño de algoritmos iterativos

Ejemplo: Bandera holandesa:

```
Tuple dutchFlag(List<E> ls, E p) {  
    n = |ls|;  
    (a, b, c) = (0, 0, n);  
    while (b < c) {  
        if (ls[b] < p) {  
            swap(ls, a, b);  
            a++;  
            b++;  
        } else if (ls[b] == p) {  
            b++;  
        } else {  
            c--;  
            swap(ls, b, c);  
        }  
    }  
    return (a, b);  
}
```

```
it(a,b,ls) {  
    (ls[a],ls[b]) = (ls[b],ls[a]);  
}
```

Esquemas Iterativos y Recursivos con Secuencias y Acumuladores

**Análisis y Diseño de Datos y Algoritmos
Estructuras de Datos y Algoritmos**

**ESCUELA TÉCNICA SUPERIOR DE
INGENIERÍA INFORMÁTICA
Departamento de Lenguajes y Sistemas Informáticos
Curso 2026-2027**

Tuplas con record (Java)

Simulamos tuplas mediante *record*:

```
record T(Integer i, Double b) {}  
  
T t = new T(0, 0.);
```

Acceso a las propiedades mediante *t.i()*, *t.b()*.

Normalmente, la declaración se complementa de un método de factoría.

```
record T(Integer i, Double b) {  
    public static T of(Integer i, Double b) {  
        return new T(i, b);  
    }  
  
    T t = T.of(0, 0.);
```

```
public static record T(Integer a, Double b, String c) {  
    public static T of(Integer a, Double b, String c) {  
        return new T(a, b, c);  
    }  
    T t = T.of(2, 5.4, "Hola");  
    T t2 = t;  
    String s = t.c();
```